團體作業

一、基本規範

1. 選出組長、副組長。團隊每週至少進行一次實體討論，並拍攝一張團隊討論照片，作為每週團隊作業之附件一併繳交。
2. 開發出的系統採命令列介面即可，不需要建置成網頁或圖形介面。
3. 所有核心功能須以自行設計的函式完成，不得使用全域變數傳遞系統資料。
4. 每個模組均須有明確的輸入、輸出、錯誤處理及測試案例。
5. 須使用 Git 進行版本管理，並適當運用生成式 AI 進行協作學習。生成式 AI 之使用目的在於協助問題分析、程式設計、除錯、測試及反思，**不得直接以 AI 產生之內容取代團隊應完成之分析、設計、實作與驗證工作**。
6. 繳交文件含程式碼、設計文件、測試報告、AI 協作紀錄及每位成員工作紀錄。
7. 若人數少於五人，由組長協調兼任工作，每位成員須負責一個可獨立測試的核心模組。
8. 核心函式應能獨立測試，不得依賴人工輸入操作。
9. 每位成員均須提交個人工作紀錄，至少包含負責功能、完成日期、Git 提交紀錄、遇到的問題、AI 使用情況及測試結果。每週團隊會議須記錄出席、討論事項、工作分配、決議及下週目標。
10. 每個核心模組須經過至少一位非原開發者的程式碼檢視。驗收時，隨機指定任一成員解釋其他成員負責的模組，並要求修改一項小型功能或修正一個錯誤，以確認團隊知識共享的程度。
11. 本作業所列功能需求為基本需求規格；各功能中未明確規定之輸入資料、輸出格式、處理規則、錯誤處理方式及驗收條件，均屬於團隊需求分析與設計工作的一部分。各團隊應透過討論自行訂定合理且一致的規格，記錄於需求規格文件中，經團隊確認後據以設計、實作與測試。後續若修改規格，須記錄修改內容及原因。
12. 第 8 週以前之測試以功能驗證及自動化測試為主；自第 10 週起，新增核心功能須進一步採 TDD 方式進行開發。

Use case model 簡介教學 https://youtu.be/2Fm3FRJqsCk

TDD 簡介教學 https://

二、具體功能需求

每項均列出輸入資料、處理規則、輸出結果及驗收條件。

F1. 課程與學生資料管理

(1) 輸入資料：課程包含課程代號、名稱、學分、必選修、授課教師、修課人數上限、教室及每週上課時段。學生包含學號、姓名、系級、已選課程及學分上限。

(2) 功能規則

- 支援課程與學生資料的新增、修改、刪除及查詢。
- 課程代號及學號不得重複。
- 每門課程可包含多個上課時段，時段以星期 1–7 及節次 1–9、a、b、c 表示。
- 學分必須為正整數，課程人數上限不得小於 1。
- 不得刪除已有學生選修的課程，除非先執行退選或課程取消程序。
- 支援依課程代號、教師、星期、必選修及剩餘名額進行查詢。

(3) 輸出結果範例

(4) 驗收條件：至少建立 20 門課程、15 名學生及 5 間教室的測試資料。系統須能正確拒絕重複代號、非法節次及無效學分。

F2. 多層次衝堂與選課檢查

(1) 輸入資料：學號、欲加選課程代號，以及目前已選課程、活動與教室配置資料。

(2) 功能規則

- 學生衝堂：同一名學生不得選修上課時段重疊的課程。
- 教室衝堂：同一教室在同一時段不得安排兩門課程或活動。
- 活動衝突：檢查學生欲參加的活動是否與已選課程或其他活動重疊。
- 學分限制：加選後總學分不得超過學生設定的上限。
- 人數限制：若課程已額滿，系統須拒絕加選。
- 必須檢查所有衝突，而不是發現第一項錯誤後立即停止。

(3) 輸出結果範例

學生：S001

欲加選：CS102 Python 程式設計

選課結果：失敗

衝突 1：星期二第 3、4 節

已選課程：MA101 微積分

衝突 2：星期四第 5 節

已參加活動：程式設計讀書會

目前學分：18

加選後學分：21

學分上限：24

(4) 驗收條件：系統必須一次列出所有衝突，依星期及節次排序。不得因某項條件通過，就忽略其他限制。

F3. 個人課表產生與空閒時段查詢

(1) 輸入資料：

(2) 功能規則

- 將學生已選課程及活動轉換為每週二維課表。
- 支援依星期、節次及課程代號查詢。
- 查詢指定時間範圍內的連續空閒時段。
- 統計每日上課節數、每週總學分及空堂數。
- 顯示不同課程及活動的時段占用情況。
- 同一課程若每週上課多次，須在所有對應時段正確顯示。

(3) 輸出結果範例：

(4) 驗收條件：以星期一至星期日及 12 個節次建立課表矩陣，並測試連續三節空堂、跨日查詢及同一天多門課程的情況。

F4. 校園活動與教室配置

(1) 輸入資料：活動代號、名稱、日期、開始與結束時間、預計人數、活動類型及場地需求。

(2) 功能規則

- 支援活動新增、取消、修改及查詢。
- 依教室容量、可用時段及設備需求篩選合適場地。
- 同一教室不得在相同時段重複配置。
- 活動報名人數不得超過活動或教室容量。
- 若無合適教室，系統須顯示未滿足的條件。
- 教室配置須與課程資料共用同一套時段衝突檢查機制。

(3) 輸出結果範例：

(4) 驗收條件：至少模擬 10 項活動，包含多人報名、活動取消、教室額滿及場地衝突等情況。

F5. 多條件課程篩選與可行性檢查

(1) 輸入資料：

(2) 功能規則

- 學生可以輸入希望修習的課程、必修清單、最低及最高學分、不可上課時段，以及偏好的上課星期。
- 系統須從課程資料中篩選符合基本條件的候選課程。

(3) 輸出結果範例：

(4) 驗收條件：至少建立五種不同限制條件的選課情境，包含條件互相矛盾、課程額滿及無法滿足最低學分等情況。

F6. 資料儲存與系統測試

(1) 輸入資料：

(2) 功能規則

- 系統須將課程、學生、教室、活動及選課紀錄儲存為 CSV 或 JSON 檔案。

- 重新啟動程式後，應能載入原有資料，且不得因單一筆資料格式錯誤造成整個程式異常終止。
- 至少設計 30 個自動化測試案例，其中正常案例不少於 10 個、邊界案例不少於 10 個、錯誤及衝突案例不少於 10 個。

(3) 輸出結果範例：

(4) 驗收條件：

F7. 多條件最佳課表搜尋

(1) 輸入必修課、候選選修課、學分限制、不可上課時段及偏好條件後，系統須搜尋所有可行課表，並依自訂評分函式排序。

(2) 必要條件包括：不得衝堂、必修課必須全部納入、總學分須介於上下限之間、不得選擇額滿課程，以及不得安排在禁止上課時段。

(3) 偏好條件可以包含：減少每週到校天數、減少空堂、避免早八課程、集中上課時段及提高偏好課程的選課比例。

(4) 自行定義評分公式及權重，說明不同權重如何影響推薦結果。

F8. 搜尋演算法比較

(1) 必要實作

- 窮舉搜尋：產生所有候選課程組合，逐一檢查限制條件。
- 回溯搜尋：在搜尋過程中及早排除違反限制的組合。
- 記錄兩種演算法的搜尋節點數、執行時間及可行解數量。
- 驗證兩種方法對相同輸入是否產生一致的可行解集合。
- 比較不同候選課程數量對執行時間的影響。

(2) 驗收條件：至少設計候選課程數量為 8、12 及 16 門的三組測試資料。若窮舉法執行時間過長，須設定合理的時間上限，並如實記錄未完成情況。

F9. 校園移動路徑規劃

(1) 以校園建築物作為節點，建築物之間的步行路線作為邊，建立校園地圖。

(2) 自行設計至少 10 個建築物節點及 15 條路線。若各路線移動成本相同，可使用廣度優先搜尋（BFS）；若各路線具有不同的步行時間，則須使用 Dijkstra 演算法，計算兩個地點之間的最短移動路徑或最短移動時間。

(3) 將移動時間納入課表可行性檢查。例如，前一堂課結束後只有 10 分鐘，而下一堂課的最短移動時間需要 15 分鐘，系統應將此安排標示為不可行。

F10. 動態事件與課表重新規劃

(1) 系統須模擬至少四種事件：

- 課程額滿，學生必須改選其他課程。
- 教師更改上課時間，造成原有課表衝突。
- 教室臨時停用，必須重新安排教室。
- 學生新增必須參加的活動，導致原有課表不再可行。

(2) 發生事件後，系統須重新檢查所有必要條件，保留仍然有效的課程安排，並產生至少一項可行的替代方案；若不存在可行方案，須明確指出原因。

F11. AI 輔助測試與程式品質改善

(1) 選至少三個核心模組，利用生成式 AI 協助分析程式結構、找出潛在錯誤及提出改善建議。

(2) 每個模組至少進行兩輪 AI 協作，記錄原始程式、提示內容、AI 回覆、修改理由及測試結果。不得直接採納未經驗證的 AI 建議，也不得將 AI 產生的程式視為已通過測試。

(3) 提交程式品質改善報告，說明哪些建議有效、哪些建議錯誤，以及如何透過測試與推理做出判斷。

三、建議團隊組織與工作分配方式 (可自行變更規劃)

建議採用「固定模組負責人、階段性角色輪替」制度。每人對一個核心模組負主要責任，但不得只參與自己的模組；所有成員均須參與需求分析、設計審查、測試及 AI 協作反思。A 為組長，副組長由 B～E 中選一人兼任，負責組長缺席時協調與進度追蹤。

成員 A：專案經理與系統整合負責人

(1) 負責建立專案工作分解結構、安排每週進度、制定資料格式及函式介面、管理 Git 版本，並整合各模組為可執行的完整系統。

(2) 負責管理功能擴充、協調模組介面修改、處理整合衝突、追蹤系統缺陷，以及確認功能未因擴充而失效。

(3) 交付成果： 專案進度表、系統架構圖、模組介面文件、整合紀錄、最終執行說明。

成員 B：需求分析與資料管理負責人

(1) 負責使用者訪談、需求規格、Use Case、課程與學生資料模型，以及 F1 資料管理模組。

(2) 負責擴充課程偏好及限制條件的資料結構，建立最佳課表搜尋所需的測試資料，並協助設計動態事件的資料更新機制。

(3) 交付成果： 使用者需求報告、Use Case、資料模型、資料管理程式、資料驗證測試。

成員 C：課程規劃與演算法負責人

(1) 負責 F2 衝堂檢查及 F5 多條件課程篩選，設計可重複使用的時間衝突判斷函式。

(2) 負責 F7 最佳課表搜尋及 F8 演算法比較，實作窮舉與回溯搜尋，並進行效能實驗。

(3) 交付成果： 演算法設計文件、衝突檢查程式、搜尋演算法、效能比較報告。

成員 D：課表、活動與路徑規劃負責人

(1) 負責 F3 個人課表產生及 F4 活動與教室配置，建立二維課表及活動時段管理機制。

(2) 負責 F9 校園移動路徑規劃，並與成員 C 合作，將移動時間限制納入課表搜尋。

(3) 交付成果： 課表產生程式、活動管理模組、校園圖形資料、路徑搜尋程式及測試報告。

成員 E：測試、品質與 AI 協作負責人

(1) 負責建立自動化測試架構、設計測試案例、管理缺陷紀錄，以及彙整各成員的 AI 協作資料。

(2) 負責 F10 動態事件模擬及 F11 程式品質改善，執行整合測試、回歸測試及 AI 建議驗證。

(3) 交付成果： 測試計畫、測試程式、缺陷追蹤表、AI 協作分析報告、品質改善紀錄。

* 上述角色為主要負責角色，而非唯一開發者。各功能至少應有一名主要負責者及一名協作者或 Reviewer；團隊成員不得僅完成自己負責的模組而未參與其他模組之需求討論、設計、測試或 Code Review。

四、團隊作業任務時程

週次	開發功能	團隊任務	團隊繳交	個人繳交
5/ Iteration 0	專案啟動	分析使用者需求；確定角色；定義資料格式；建立 Git Repository；建立 Coding Convention；規劃 F1～F11 依賴關係	需求規格 v1、Use Case、系統架構 v1、資料格式、工作分解表、Git Repository	個人角色與負責工作說明、個人本週工作紀錄
6/ Iteration 1	F1	完成課程、學生及教室基本資料管理；建立第一個可執行版本；進行第一次 Code Review	SRS、F1 程式、資料模型、函式介面、測試案例、Iteration Review 1	個人程式碼、Git Commit 紀錄、Code Review 紀錄、AI 使用紀錄
7/ Iteration 2	F2＋F3，並開始 F4 之需求分析與介面設計	在 F1 上增加衝堂檢查、選課檢查、課表產生及空閒時段查詢；不得重新建立另一套獨立資料	可執行 v2、F2/F3 程式、測試案例、缺陷紀錄、Iteration Review 2	負責模組說明、測試結果、Review 其他成員程式紀錄
8/ Iteration 3	F4 ＋ F5 ＋ F6	整合活動與教室、條件式課程篩選、CSV/JSON 資料儲存；對 F1～F6 執行整合測試及回歸測試	**期中系統 v1.0**、完整程式、30 個以上自動化測試、測試報告、AI 協作紀錄、影片 I	個人工作報告 I、Git 紀錄、個人負責功能說明、個人 AI 協作反思 I
9	**第一次上機測驗**	個人能力評量	無	**第一次上機測驗**
10/	F7	重新檢視期中架構；定義最佳	F7 需求規格、TDD	個人 TDD 紀錄：至少

Iteration 4		課表需求及評分函式；正式導入 TDD；先設計 F7 測試，再實作搜尋功能	測試程式、Red/Green/Refactor 紀錄、系統 v1.1	一個 Red→Green→Refactor 完整案例
11/ Iteration 5A	F8	實作窮舉及回溯搜尋；使用相同測試資料驗證結果一致性；比較 8、12、16 門候選課程	F8 程式、演算法說明、TDD 紀錄、效能實驗初稿	個人演算法說明、測試設計及 Git 紀錄
12/ Iteration 5B	F9	建立校園 Graph；以測試先定義最短路徑預期結果；完成 BFS/Dijkstra；將移動時間整合至 F7	F9 程式、Graph 資料、TDD 紀錄、F7＋F8＋F9 整合版 v1.2	個人測試案例、Code Review 及問題修正紀錄
13/ Iteration 5C	F7 ＋ F8 ＋ F9 整合	將課程限制、偏好評分、搜尋演算法及移動時間完整整合；執行回歸測試	系統 v1.3、整合測試報告、效能比較報告 v1	個人 Iteration 反思、負責模組改進紀錄
14/ Iteration 6A	F10	先針對四種動態事件設計失敗測試，再實作重新規劃；檢查重新規劃是否盡量保留原課表	F10 程式、四類事件測試、TDD 紀錄、系統 v1.4	個人 Red→Green→Refactor 紀錄及錯誤修正說明
15/ Iteration 6B	F11	選至少 3 個核心模組進行 AI 輔助測試與品質改善；完成人工測試與 AI 測試比較；開始 F1～F11 整合及回歸測試	AI 協作紀錄、程式品質改善報告初稿、AI 測試與人工測試比較表、整合問題清單	至少一個 AI 正確建議及一個 AI 錯誤／不適當建議的驗證紀錄
16	**第二次上機測驗**	個人能力評量	無	**第二次上機測驗**
17/ Final Iteration	F1～F11 全系統	完成全系統整合、Regression Testing、跨模組錯誤修正、重新執行 F1～F11 測試、效能實驗、程式重構、文件與 AI 協作歷程整理，製作影片 II	最終程式 v2.0、完整回歸測試報告、缺陷與修正紀錄、專題報告、影片 II	個人工作報告 II、個人貢獻說明、AI 協作反思 II、Code Review 紀錄、個人負責缺陷修正紀錄
18/ Final Review	最終驗收	系統展示、隨機口頭詢問、跨模組程式解釋及現場功能修改	最終修正版程式與文件、現場程式修改及同儕互評	

五、生成式 AI 協作學習經驗／心得數位分享影片規範

影片 I：從需求分析到可執行系統——我們如何與生成式 AI 協作 (長度：6～8 分鐘)

影片不得只是展示系統功能，或只介紹「我們用了 ChatGPT 做什麼」，而須選擇 F1～F6 開發過程中**一個實際且具代表性的問題**，完整呈現團隊解決問題與 AI 協作的歷程，**五位成員原則上均須實際參與說明**。至少包含：

I-1 問題情境：說明遇到什麼程式設計問題，為什麼此問題不容易直接解決。

I-2 問題分解：說明如何利用設計思考或功能分解，將複雜問題拆成較小問題。

I-3 原始解法：呈現團隊在詢問 AI 以前的想法、演算法或程式。

I-4 AI 協作過程：呈現至少兩輪 Prompt 及 AI 主要建議，說明為什麼第二輪 Prompt 需要修改。

I-5 AI 建議的驗證：至少呈現一項「AI 提供的建議不能直接採用」的例子，說明團隊如何透過程式執行、測試案例或邏輯分析進行驗證。

I-6 修改前後比較：比較 AI 協作前後的演算法、程式碼或測試結果。

I-7 團隊反思：說明 AI 在哪些工作上有幫助、哪些工作仍必須由學生自己判斷。

影片 II：從 AI 產生程式到 AI 協作驗證——複雜問題、TDD 與程式品質(長度：8～10 分鐘)

II-1 複雜問題說明：從 F7～F10 選擇一項具有代表性的複雜問題，例如最佳課表搜尋、Backtracking、最短路徑或動態重新規劃。

II-2 解題策略比較：至少比較兩種可能的解法，說明最後採用某種方法的理由。

II-3 TDD 實際案例：實際呈現一次 Requirement → Test → Red → Green → Refactor → Regression Test 之完整過程，不能只口頭解釋 TDD。

II-4 AI 協作過程：呈現 AI 如何協助演算法設計、除錯、測試案例或重構。

II-5 AI 錯誤與驗證：至少呈現一個 AI 產生的：錯誤程式；不完整測試；不適用演算法；或看似合理但實際不正確的建議；並說明如何發現。

II-6 實驗證據：利用實際數據說明改善結果，例如：測試通過率；搜尋節點數；執行時間；缺陷數；Regression Test 結果。

II-7 與影片 I 比較：比較期中與期末在「問題分解方式、Prompt 設計方式、AI 建議驗證方式及學生自主判斷」四個面向的差異，並回答：期中時我們如何使用 AI？期末時我們如何使用 AI？我們的 AI 協作方式有什麼改變？

II-8 個人反思：每位成員以約 20～30 秒回答：「如果沒有 AI，我會如何解決這個問題？」「AI 最能幫助我的地方是什麼？」「AI 最不能取代我的能力是什麼？」

影片附件： 除影片檔案或影片連結外，團隊須一併繳交影片中使用之 Prompt、程式碼版本、測試案例及實驗數據等佐證資料。影片中呈現之 AI 協作內容須能由繳交資料追溯，不得僅以口頭敘述代替實際紀錄。

六、每週團隊繳交規範

(1) 本週系統版本：可以直接執行之 Python 程式。

(2) 功能完成表：標示 F1～F11 各功能目前狀態為「未開始、開發中、已完成、測試通過」。

(3) 需求／設計修改紀錄：說明本週相較前一版本修改哪些需求、資料結構或函式介面，以及修改原因。

(4) 自動化測試程式與結果：不得僅以人工操作畫面作為測試證據。

(5) Git 紀錄：提供本週主要 Commit、Merge 及負責成員。

(6) Code Review 紀錄：至少一項功能由非原開發者進行 Review。

(7) 問題與缺陷紀錄：說明本週發現之問題、處理方式及尚未解決事項。

(8) Iteration Review：以一頁為原則，回答「完成什麼、未完成什麼、遇到什麼問題、下一 Iteration 如何改善」。

(9) TDD 紀錄(第 10 週起): 第 10 週起之新增核心功能,須保留至少一組「需求(Requirement) → 測試案例(Test)→ 測試失敗(Red)→ 最低限度實作(Green)→ 程式重構(Refactor) → 回歸測試 (Regression Test)」之完整紀錄。不得於功能實作完成後才補寫測試案例作為 TDD 紀錄。

七、每週個人繳交規範

(1) 個人工作紀錄：說明本週負責工作、實際完成內容、遭遇問題及解決方式。

(2) Git 紀錄：列出本人本週主要 Commit 及其功能說明；Commit 數量不作為個人成績之直接依據。

(3) 程式理解說明:選擇本週參與的一個重要函式,以自己的文字說明其 Input、Process、Output 及 Boundary/Error Case。

(4) Code Review 紀錄:每位成員至少檢視一項非本人主要負責的程式,提出問題或改善建議。

(5) AI 協作紀錄：若使用生成式 AI，應記錄問題、Prompt、AI 建議摘要、是否採納、採納或拒絕理由及驗證方式。不得僅繳交完整 AI 對話紀錄。

(6) TDD 紀錄：第 10 週起，每位成員至少須完成一項自己實際參與之 Requirement → Test → Red → Green → Refactor → Regression Test 完整紀錄。